

Flujos de Trabajo Colaborativo

Del dominio técnico individual a la ingeniería de software colaborativa. Coordinar talento humano y flujo de código para que la velocidad de desarrollo no comprometa la estabilidad del producto.

Made with GAMMA

Estrategias de Branching

Un conjunto de reglas que definen cuándo se crean ramas, cómo se nombran, con qué frecuencia se integran y hacia dónde fluyen los cambios.



Aislar el trabajo

Desarrolladores trabajan en nuevas funcionalidades sin romper la versión en producción.



Integración Continua

Puntos claros donde el código debe ser probado y validado.



Estabilizar versiones

Controlar qué código está listo para desplegarse.

Made with GAMMA

GitFlow: El Estándar Estructurado

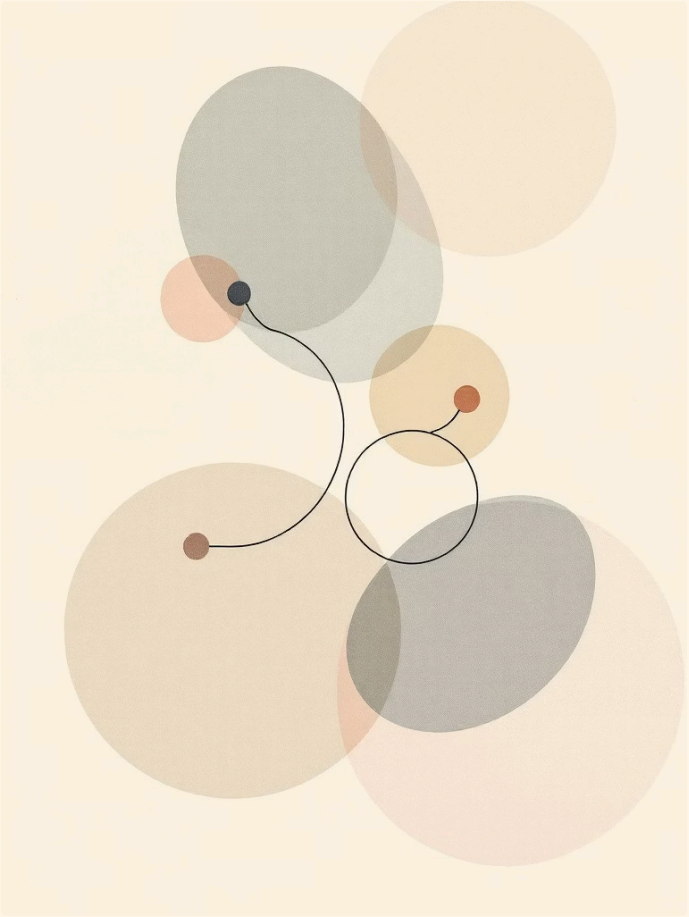
Diseñado para proyectos con ciclos de lanzamiento programados. Modelo robusto con dos ramas principales de larga duración.

Ramas principales

- **master/main:** Solo código en producción con Tags
- **develop:** Rama de integración para próximas versiones

Ramas temporales

- **feature/:** Nuevas funcionalidades
- **release/:** Ajustes finales
- **hotfix/:** Arreglos urgentes



Made with GAMMA

Flujos Ágiles: GitHub Flow y Trunk-based

GitHub Flow

Simplificado para despliegue continuo. Solo una rama principal (main). Pull Request para cada cambio, merge inmediato tras aprobación y tests.

GitLab Flow

Añade ramas para distintos entornos (Staging, Pre-producción, Producción). Flujo unidireccional: main → production.

Trunk-based Development

Modelo de alto rendimiento. Integración en main al menos una vez al día. Requiere tests automáticos sólidos y Feature Flags.



Made with GAMMA

¿Qué Modelo Elegir?

La elección depende de cuatro dimensiones críticas que impactan la operatividad del equipo.



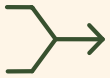
Tamaño y estructura del equipo

Jerárquico vs. colaborativo vs. sincronizado



Frecuencia de entrega

¿Qué tan rápido necesita el mercado?



Complejidad de fusión

Costo oculto de cada estrategia



Requisitos técnicos

Automatización y cultura del equipo

Made with GAMMA

Tabla Comparativa de Modelos

Factor	GitFlow	GitHub Flow	Trunk-based
Tamaño del Equipo	Equipos grandes / Jerárquicos	Equipos pequeños / Medianos	Equipos Senior / Ágiles
Frecuencia de Entrega	Semanal / Mensual	Diaria	Continua (múltiples veces al día)
Complejidad de Fusión	Alta (muchos conflictos al final)	Baja	Mínima (se integra constante)
Requisito Técnico	Disciplina de ramas	Cultura de PRs	Tests automáticos obligatorios

Made with GAMMA



Pull Requests y Code Reviews

Anatomía de un PR Profesional

- Título claro que resume el cambio
- Descripción detallada del qué, cómo y porqué
- Checklist de validación
- Evidencia: capturas o logs

Code Reviews: Filtro de Calidad

Conversación técnica para asegurar código mantenible, seguro y que siga estándares del equipo.

¿Qué buscar? Legibilidad, principios SOLID/DRY, seguridad y performance.

Made with GAMMA

Estrategias de Integración

Al aprobar un PR, el equipo decide cómo integrar el trabajo en la rama principal.



Merge Commit

Mantiene todos los commits. Prioriza trazabilidad total.



Squash and Merge

Un solo commit. Prioriza legibilidad de main.



Rebase and Merge

Historial lineal. Prioriza progresión perfecta.

Made with GAMMA

Estandarización de Documentación

01

Guías de Contribución

Manual de bienvenida: configuración, estilo de código, flujo de Git y requisitos de PR.

02

Plantillas de Trabajo

Issues y PRs predefinidos para evitar reportes incompletos y asegurar contexto.

03

ADRs (Architecture Decision Records)

Documentan decisiones técnicas importantes, contexto y consecuencias.



Made with GAMMA



Garantía de Estándares: Doble Validación

Validación Local (Git Hooks)

Primera línea de defensa en la máquina del desarrollador.

- Linters y formateadores (ESLint, Prettier)
- Git Hooks (Husky) en eventos de Git
- pre-commit bloquea commits con errores

Validación Remota (Status Checks)

Barrera final en la nube durante el Pull Request.

- Git como disparador (Trigger)
- GitHub Actions ejecuta tests
- Gatekeeping protege la rama principal

Made with GAMMA